

VIETNAM NATIONAL UNIVERSITY, HO CHI MINH CITY
UNIVERSITY OF SCIENCE
FACULTY OF INFORMATION TECHNOLOGY

[Student Name 1] - [Student Name 2]
22125027 - 22125037

uMatter: A Microservices- Based Mental Health Sup- port Platform for Teenagers

GRADUATION THESIS
Software Engineering

Advisor: [Advisor Name]

Ho Chi Minh City, 2026

Statement of Originality

I hereby declare that this is my own research, carried out under the supervision of ... The data and results presented in this thesis are truthful and have not been duplicated from any other work.

Statement on Topic Revisions

Statement describing any revisions made after the defense (e.g., changes to the thesis title or report content), if applicable.

Advisor's Comments

The advisor's signed comment sheet, provided by the academic affairs office.

Reviewer's Comments

The reviewer's signed comment sheet, provided by the academic affairs office.

Acknowledgments

We would like to sincerely thank our advisor for their guidance throughout this thesis, the Faculty of Information Technology at the University of Science (VNU-HCM) for the resources and feedback that shaped this work, and our families and friends for their continued support during the development of uMatter.

Thesis Proposal

The approved thesis proposal, previously submitted to the academic affairs office (*must bear the advisor's signature*).

Table of Contents

Statement on Topic Revisions	ii
Advisor’s Comments	iii
Reviewer’s Comments	iv
Acknowledgments	v
Thesis Proposal	vi
Table of Contents	vii
List of Figures	x
List of Tables	xi
Abstract	xi
1 Introduction	1
1.1 Background and Motivation	1
1.2 Problem Statement	2
1.3 Objectives	3
1.4 Scope and Contributions	4
1.5 Thesis Organization	4
2 Related Work	6

2.1	Self-Care and Tracking Applications	6
2.2	Professional Therapy Platforms	7
2.3	Positioning of uMatter	7
2.4	Architectural Foundations	8
2.5	Security Foundations	8
2.6	AI Companion	9
3	System Analysis and Design	11
3.1	Overall Architecture	11
3.2	Service Decomposition	12
3.3	Data Architecture	13
3.4	Authentication, Authorization, and Data Sharing	13
3.5	AI Companion	14
3.6	Professional Workflows	14
3.7	Event-Driven Messaging	15
3.8	Deployment Topology	15
4	Implementation and Evaluation	16
4.1	Implementation Overview	16
4.2	Security Hardening	17
4.3	Deployment	18
4.4	Evaluation	19
4.4.1	Functional Completeness	19
4.4.2	Security Posture	19
4.4.3	Comparison with Existing Platforms	19
5	Conclusion and Future Work	21
5.1	Conclusion	21
5.2	Limitations	22
5.3	Future Work	22
	List of Author's Publications	24

References	25
A API Endpoint Summary	27
B Deployment Configuration Notes	29

List of Figures

List of Tables

2.1	Feature comparison between uMatter and comparable platforms	10
3.1	uMatter backend services and their responsibilities	12
A.1	Representative endpoints by service	28

Abstract

Adolescent mental health support remains fragmented: self-care tools such as mood or sleep trackers rarely connect to licensed professional care, while therapy platforms such as BetterHelp, Wysa, Bearable, Daylio, or Talkspace typically specialize in only one side of that spectrum and are seldom designed around the specific needs of teenagers. This gap motivates a platform that unifies everyday self-tracking with a credible path to professional support for a teenage user base.

This thesis presents uMatter, a mental health support platform for teenagers that combines self-care features – mood, sleep, and journaling tracking, an AI companion, and peer social features – with professional therapy features, including therapist discovery and matching, appointment booking, video consultation, and clinical note-taking for licensed therapists.

uMatter is implemented as a microservices system: seven Spring Boot services (authentication, tracking, AI, dashboard, therapist, social, and notification) sit behind an Nginx API gateway, communicate asynchronously through RabbitMQ, and follow a database-per-service pattern backed by PostgreSQL, Redis, and MinIO. Access control uses stateless JWT authentication with a JWKS-published RSA key and a permission-based data-sharing model so users can selectively grant professionals access to their tracked data. The AI companion is grounded in each user’s own tracking history and served through Google’s Gemini 2.5 Flash. The system was hardened through structured security audits covering injection risks, insecure direct object references, and mass assignment, and a database-level partial unique

index was introduced to close a race condition in appointment booking. The full stack was deployed and validated on cloud infrastructure, running as roughly two dozen containers across the constituent services.

The resulting platform demonstrates that a self-care and professional-therapy experience can be delivered from a single, security-conscious microservices architecture, and it establishes a concrete foundation – clinical collaboration requirements, layered risk-detection, and further safety-oriented features – for future work extending uMatter toward broader adolescent mental health support.

Chapter 1

Introduction

1.1 Background and Motivation

Adolescent mental health is a significant and growing public-health concern: the World Health Organization estimates that one in seven adolescents worldwide experiences a mental disorder, and mental health conditions account for a substantial share of the overall disease burden in this age group [1]. Despite this, teenagers face specific barriers to care that are not fully addressed by the tools currently available to them.

On one side of the market, self-care applications such as Daylio and Bearable let users log their mood, sleep, and daily activities, helping them notice patterns in their own well-being [2], [3]. These tools are lightweight and approachable, but they stop at self-observation: they do not connect a user who needs help to a licensed professional.

On the other side, therapy-oriented platforms such as BetterHelp and Talkspace connect users with licensed therapists for text, audio, or video counseling [4], [5], while AI-first platforms such as Wysa offer a conversational companion alongside optional access to coaches [6]. These platforms are built primarily around an adult clientele and around one side of the care journey – either self-tracking or professional therapy – rather than both, and few are designed with the specific privacy, guardianship, and engagement needs of teenagers in mind.

This gap – between everyday self-tracking and a credible, integrated path into professional care – is the problem this thesis addresses.

1.2 Problem Statement

Concretely, this thesis addresses the following problem: *how can a single platform give teenagers an approachable, private way to track their own mental well-being, while also giving them and their guardians a controlled, permission-based path to matched professional support, without requiring the sensitive data collected for self-care to be exposed by default?*

This problem is difficult for several reasons:

- **Data sensitivity.** Mood, sleep, and journal data are highly personal; any sharing with a professional must be explicit, scoped, and revocable rather than all-or-nothing.
- **Heterogeneous functionality.** Self-care tracking, an AI companion, social features, and professional workflows (matching, scheduling, video consultation, clinical notes) are functionally very different and are best built, scaled, and secured as separate concerns rather than as one monolith.
- **Safety.** A platform that surfaces user-reported mental-health signals carries a responsibility to handle at-risk situations appropriately, which in turn requires clinical input that is out of scope for a single software thesis but must be designed for as a future extension point.
- **Trust and correctness.** Because the platform mediates real appointments and real personal data, common web-application vulnerability classes (broken access control, insecure direct object references, mass assignment, race conditions on booking) must be identified and closed rather than left as accepted risk [7].

1.3 Objectives

The objective of this thesis is to design, implement, and evaluate uMatter, a mental-health support platform for teenagers that combines self-care tracking, an AI companion, and peer social features with professional therapist discovery, matching, booking, video consultation, and clinical note-taking, built on a microservices architecture with a permission-based data-sharing model between users and professionals.

This overall objective decomposes into the following concrete goals:

1. Design a service decomposition and data architecture (database-per-service, asynchronous eventing) appropriate for the platform’s distinct functional areas.
2. Implement authentication and authorization (stateless JWT with a JWKS-published key) and a permission-based data-sharing model so that tracked data is not exposed to professionals by default.
3. Implement the self-care experience (mood/sleep/journal tracking, an AI companion grounded in the user’s own data, and peer social features) and the professional experience (therapist discovery and matching, appointment booking, video consultation, clinical notes).
4. Harden the system against common vulnerability classes through structured security audits, and resolve at least one concrete data-integrity issue (a race condition on appointment booking).
5. Deploy the full stack to cloud infrastructure and evaluate the resulting system against its design goals and against comparable existing platforms.

1.4 Scope and Contributions

This thesis is a software engineering thesis: it does not propose or validate a clinical intervention, and any features that would require clinical judgment (for example, automated risk detection acting on self-reported data) are treated as future work requiring collaboration with mental-health professionals, not as deliverables of this thesis.

The main contributions of this work are:

- A working microservices architecture for a combined self-care and professional-therapy platform, covering seven backend services, a mobile client for teenagers, and a web client for therapists.
- A permission-based data-sharing design that lets a user grant a specific professional scoped access to their tracked data, rather than exposing all data to all connected professionals.
- A structured security-audit process applied to the implemented system, including the identification and resolution of an appointment-booking race condition using a database-level partial unique index.
- A cloud deployment of the full stack and an evaluation of the platform against comparable products in the market.

1.5 Thesis Organization

The remainder of this thesis is organized as follows. Chapter 2 reviews related work, comparing uMatter against existing self-care and therapy platforms and identifying the feature and design gaps this thesis addresses. Chapter 3 presents the proposed system architecture and design, covering the service decomposition, data architecture, authentication and data-sharing model, and the AI companion. Chapter 4 describes the implementation, the

security-hardening process, the cloud deployment, and the evaluation of the resulting system. Chapter 5 concludes the thesis and discusses future work.

Chapter 2

Related Work

This chapter surveys existing digital mental-health platforms, positions uMatter relative to them, and reviews the architectural and security literature that informs the design presented in Chapter 3.

2.1 Self-Care and Tracking Applications

Daylio and Bearable are representative of a category of lightweight, single-purpose self-care applications [2], [3]. Daylio focuses on quick, low-friction mood and activity logging, encouraging habit formation through streaks and reminders. Bearable extends this idea to a broader set of symptoms and factors (sleep, medication, exercise, weather, and more), letting users explore correlations in their own data.

Both applications are valuable for self-observation, but neither connects a user to a professional: there is no booking, matching, or clinical workflow, and the data collected has no path out of the app other than manual export. For a teenager who identifies a concerning pattern in their own tracked data, these tools offer no next step.

2.2 Professional Therapy Platforms

BetterHelp and Talkspace sit at the opposite end of the spectrum [4], [5]. Both connect users with licensed therapists through a subscription model, primarily via asynchronous messaging with optional live sessions. Their onboarding is built around a short intake questionnaire used to match a new user to a therapist, and their core workflows (scheduling, secure messaging, video sessions) are mature and well-tested at scale.

These platforms are built for a general adult audience, and they do not provide day-to-day self-care tracking as a first-class feature – a user’s mood or sleep history is not something the platform helps them build up over time outside of a therapy session. They also do not target the specific constraints of a teenage user base, such as guardian involvement or age-appropriate content and matching.

Wysa occupies a middle position: it offers an AI conversational companion for everyday emotional support, with an optional paid tier that adds access to human coaches [6]. This is architecturally closer to uMatter’s combination of an AI companion with professional access, but Wysa’s professional tier is coaching-oriented rather than built around licensed-therapist matching, booking, and clinical note-taking as core, tightly integrated workflows.

2.3 Positioning of uMatter

Table 2.1 summarizes this comparison. uMatter’s distinguishing position is that it treats self-care tracking and professional therapy as two halves of one continuous user journey, connected by a permission-based data-sharing model: a user’s own tracked history can be selectively shared with a matched therapist, rather than being siloed in a separate self-care app or absent from a therapy-only platform.

2.4 Architectural Foundations

The system architecture proposed in Chapter 3 builds on well-established distributed-systems practice rather than novel architectural research. The decomposition into independently deployable services, each owning its own data store, follows the microservices style described by Newman [8], chosen over a monolithic design because uMatter’s functional areas (authentication, tracking, AI, professional workflows, social features, notifications) have different scaling, security, and change-rate profiles. Inter-service communication uses a mix of synchronous REST calls, following the architectural constraints of the REST style [9], and asynchronous messaging through RabbitMQ for event-driven workflows such as notifications [10]. Services are implemented with Spring Boot [11] and packaged with Docker [12] to keep the development and deployment environments consistent.

2.5 Security Foundations

Because uMatter mediates sensitive personal data and real professional appointments, its security design draws directly on two established references. Authentication uses stateless JSON Web Tokens as defined in RFC 7519 [13], verified against a JWKS-published public key so that individual services can validate tokens without a shared session store. The security-audit process described in Chapter 4 is organized around the OWASP Top 10 risk categories [7], with particular attention to broken access control (insecure direct object references), injection, and data-integrity issues such as race conditions – categories that are directly relevant to a system that stores personal health-adjacent data and coordinates real-world appointments.

2.6 AI Companion

The in-app AI companion is built on Google’s Gemini 2.5 Flash model [14]. Unlike a general-purpose chatbot, the companion is grounded in the user’s own tracking history, so its responses can refer to a user’s recently logged mood, sleep, or journal entries rather than operating from a blank context. Video consultation between users and therapists is provided through Jitsi Meet [15], an open-source video-conferencing solution that avoids dependence on a proprietary video vendor.

Table 2.1: Feature comparison between uMatter and comparable platforms

Feature	uMatter	Better- Help / Talkspace	Wysa	Daylio / Bear- able
Mood/sleep- /journal tracking	Yes	No	Partial	Yes
AI companion grounded in own data	Yes	No	Yes (general)	No
Licensed therapist matching & booking	Yes	Yes	Limited	No
Video consultation	Yes	Yes	No	No
Clinical notes for professionals	Yes	Yes	No	No
Peer social features	Yes	No	No	No
Teen-oriented design	Yes	No	Partial	Partial
Granular, permission- based data sharing	Yes	N/A	N/A	N/A

Chapter 3

System Analysis and Design

This chapter presents the proposed architecture for uMatter: its overall topology, service decomposition, data architecture, event-driven messaging, and security and data-sharing model.

3.1 Overall Architecture

uMatter follows a microservices architecture: independently deployable Spring Boot services sit behind a single Nginx API gateway, which is the only entry point reachable by the two client applications – a React Native mobile app for teenagers/patients and a React web dashboard for therapists. The gateway is responsible for routing requests to the correct backend service and for terminating HTTPS at the edge of the system.

This decomposition follows the pattern described by Newman [8]: each service is scoped to a single functional area, can be developed, deployed, and scaled independently, and owns its own data. Synchronous, request/response interactions between services and clients use REST-style HTTP APIs [9]; workflows that are naturally asynchronous, such as sending a notification after an event happens elsewhere in the system, are handled through RabbitMQ instead [10].

3.2 Service Decomposition

The backend is organized into seven services, summarized in Table 3.1.

Table 3.1: uMatter backend services and their responsibilities

Service	Responsibility
Auth	User registration/login, JWT issuance and refresh, JWKS key publication, role management
Tracking	Mood, sleep, and journal entry storage; the source of truth for a user’s self-reported well-being data
AI	Manages conversations with the AI companion, grounding responses in the user’s own tracking data
Dashboard	Aggregates tracking data into summaries and visual trends for the user
Therapist	Therapist discovery/matching, appointment booking, video consultation session management, clinical notes
Social	Peer relationships (friends) and real-time chat over STOMP/WebSocket
Notification	Consumes domain events and delivers push (FCM), email, and in-app inbox notifications

The Auth, AI, Tracking, and Dashboard services are grouped in a single multi-module backend repository, since they were developed together and share a close data-flow relationship, while the Therapist, Social, and Notification services are maintained as separate repositories, reflecting their more independent lifecycles and, in the case of Social and Notification, different Spring Boot versions.

3.3 Data Architecture

uMatter follows a database-per-service pattern: each backend service owns a private PostgreSQL database, and no service reaches directly into another service's schema. Cross-service data needs are satisfied either through a synchronous API call to the owning service or through an asynchronous event, never through shared tables. This keeps each service free to evolve its own schema without coordinating a migration across the whole system, at the cost of needing an explicit strategy (API calls or events) for any data that spans services.

Two supporting stores complement PostgreSQL:

- **Redis** is used for caching and for idempotency keys, so that a retried request (for example, a client retry after a network timeout) is not applied twice.
- **MinIO**, an S3-compatible object store, holds binary media such as profile photos and any files attached to journal entries or clinical notes, keeping large binary payloads out of the relational databases.

3.4 Authentication, Authorization, and Data Sharing

Authentication is stateless: the Auth service issues JSON Web Tokens signed with an RSA private key, following RFC 7519 [13], and publishes the corresponding public key through a JWKS endpoint. Every other service validates incoming tokens locally against this published key, so no service needs to call back to Auth (or share a session store) simply to authenticate a request. Roles embedded in the token (teen/patient, therapist, admin) drive coarse-grained authorization at the gateway and service level.

Fine-grained authorization is handled by a permission-based data-sharing model, which is central to uMatter’s design goal of not exposing sensitive tracked data by default. A user’s mood, sleep, and journal entries are private to that user unless the user explicitly grants a specific therapist scoped, revocable access – for example, access to the last 30 days of mood data, but not to journal text. This is enforced at the Tracking and Therapist service boundary rather than left to the client application, so that a compromised or misbehaving client cannot bypass the grant.

3.5 AI Companion

The AI companion is implemented in the AI service, which retrieves relevant recent entries from the Tracking service and includes them as context when calling Google’s Gemini 2.5 Flash model [14]. Grounding the model in the user’s own data lets the companion respond to something concrete (“you logged trouble sleeping three nights this week”) rather than only offering generic supportive text. The AI service does not attempt automated clinical risk detection; any escalation logic of that kind is deliberately left out of scope, as discussed in Chapter 5, pending the clinical collaboration such a feature would require.

3.6 Professional Workflows

The Therapist service implements the professional-facing half of the platform: it matches users to therapists based on stated preferences and availability, handles appointment booking, manages Jitsi Meet video-consultation sessions [15] for the appointment, and stores clinical notes that a therapist records after a session. Appointment booking is a point of particular concern for data integrity, since two concurrent booking requests for the same therapist and time slot must not both succeed; the resolution of this issue is described in Chapter 4.

3.7 Event-Driven Messaging

RabbitMQ [10] connects services that should not depend on each other synchronously. For example, when an appointment is booked by the Therapist service, an event is published describing the booking; the Notification service subscribes to this event independently and is responsible for turning it into a push notification, an email, and an in-app inbox entry, without the Therapist service needing to know anything about how notifications are delivered. This keeps the Notification service’s delivery logic (and any future delivery channel) fully decoupled from the services that generate the underlying events.

3.8 Deployment Topology

All services are packaged as Docker containers [12] and orchestrated with Docker Compose, grouped into stacks by repository. The full system – Spring Boot services, PostgreSQL instances, RabbitMQ, Redis, MinIO, and supporting tools such as pgAdmin – runs on a single cloud virtual machine, fronted by a reverse proxy that terminates HTTPS. The concrete deployment used for evaluation, including the migration between cloud providers, is described in Chapter 4.

Chapter 4

Implementation and Evaluation

This chapter describes how the design in Chapter 3 was implemented, the security-hardening process applied to it, the cloud deployment used to run and evaluate the system, and an evaluation of the resulting platform.

4.1 Implementation Overview

The backend services are implemented in Java 17. The Auth, Tracking, AI, and Dashboard services are built on Spring Boot 4.0 and organized as a Maven multi-module project so that shared code (for example, JWT validation logic) is written once and reused. The Therapist service is a separate Gradle project on Spring Boot 4.0, while the Social and Notification services are Gradle projects on the slightly earlier Spring Boot 3.3, reflecting when they were added to the system. Persistence throughout uses Spring Data JPA over PostgreSQL. The Social service's real-time chat is implemented with STOMP over WebSocket, and the Notification service integrates Firebase Cloud Messaging for push delivery in addition to email and an in-app inbox.

On the client side, the mobile application for teenagers is built with React Native 0.83 and React 19 in TypeScript, and the therapist-facing web dashboard is built with React 18, TypeScript, Vite, and Tailwind CSS.

4.2 Security Hardening

Because uMatter handles sensitive personal data and coordinates real appointments, the implemented services were subjected to structured security audits organized around the OWASP Top 10 risk categories [7], covering three areas in particular:

- **Business logic correctness** – verifying that workflows such as matching, booking, and permission granting behave correctly under normal and adversarial input, not only in the happy path.
- **Access control** – specifically JPQL injection patterns in hand-written queries, insecure direct object references (for example, a user supplying another user’s record identifier to an endpoint that does not re-check ownership), and mass assignment (a client supplying extra fields in a request body that get bound to fields the client should not be able to set, such as a role or a permission flag).
- **Data integrity** – ensuring that concurrent requests cannot leave the system in an inconsistent state.

The most significant data-integrity issue identified was a race condition in appointment booking: two concurrent requests to book the same therapist for the same time slot could both pass an application-level availability check before either write was committed, resulting in a double booking. This was closed not by adding an application-level lock, which would not be safe against multiple service instances, but by adding a database-level partial unique index on the appointment table, scoped to active (non-cancelled) appointments for a given therapist and time slot. This pushes the correctness guarantee down to the database itself, so the constraint holds regardless of how many instances of the Therapist service are running concurrently.

4.3 Deployment

uMatter was originally deployed on Oracle Cloud Infrastructure (OCI). During the course of this thesis, the deployment was migrated in full to Microsoft Azure, under an Azure for Students subscription. The migration involved:

- Resolving Azure region-policy restrictions that initially blocked the target subscription from provisioning a virtual machine in the desired region.
- Configuring Network Security Group (NSG) rules to open the ports required by the API gateway and by directly-exposed services (for example, RabbitMQ’s management port during setup).
- Working around Azure Cloud Shell setup issues encountered while preparing the target VM.
- Reproducing the auto-shutdown behavior available on the previous provider using a cron-based scheduled shutdown on the Ubuntu VM, to control cost under the student subscription.

The resulting deployment runs on a single, right-sized Azure VM hosting roughly twenty-one containers: multiple Spring Boot JVM services, several PostgreSQL instances (one per owning service), RabbitMQ, Redis, MinIO, and pgAdmin. HTTPS routing is provided by Caddy as a reverse proxy, with DuckDNS supplying a stable hostname for the VM’s dynamic address. This mirrors the general topology described in Chapter 3, with Caddy taking the reverse-proxy/HTTPS-termination role and Nginx continuing to serve as the internal API gateway between the proxy and the backend services.

4.4 Evaluation

The system was evaluated along three dimensions: functional completeness against the design goals in Chapter 1, security posture after hardening, and feature positioning relative to the competitive landscape reviewed in Chapter 2.

4.4.1 Functional Completeness

All seven backend services described in Chapter 3 are implemented and deployed, and both client applications (mobile for teens, web for therapists) are functional against them. The permission-based data-sharing model is enforced at the API level, so tracked data is not visible to a therapist who has not been explicitly granted access to it. The AI companion successfully retrieves and uses a user’s own recent tracking data as context for its responses.

4.4.2 Security Posture

The structured audits identified and closed concrete issues in each of the three areas examined (business logic, access control, and data integrity), including the appointment-booking race condition described above. This does not constitute a formal, independent penetration test, and should be read as an internal hardening pass rather than a certification of the system’s security; further external review would be a reasonable next step before any production use involving real users.

4.4.3 Comparison with Existing Platforms

Revisiting the comparison in Table 2.1, the implemented system delivers on the feature combination that motivated this thesis: self-care tracking, an AI companion grounded in the user’s own data, and peer social features,

alongside licensed-therapist matching, booking, video consultation, and clinical notes, connected by a permission-based sharing model that none of the compared single-purpose platforms offer in combination. The clearest gap relative to mature commercial platforms such as BetterHelp and Talkspace [4], [5] is operational maturity: those platforms have been hardened and scaled in production for years, whereas uMatter, as a thesis project, has been validated on a single-VM deployment and an internal security-audit process rather than at production scale.

Chapter 5

Conclusion and Future Work

5.1 Conclusion

This thesis set out to design, implement, and evaluate uMatter, a mental-health support platform for teenagers that unifies self-care tracking with professional therapy access, addressing a gap identified between existing self-care applications (Daylio, Bearable) and existing therapy platforms (BetterHelp, Talkspace, Wysa), none of which combine both halves of the care journey behind a single, permission-based data-sharing model.

The resulting system is a seven-service microservices architecture behind an API gateway, with a database-per-service data layer, stateless JWT authentication, and an AI companion grounded in each user’s own tracking data. A structured security-hardening process identified and resolved concrete issues across business logic, access control, and data integrity, including a database-level fix for a race condition in appointment booking. The full stack was deployed to cloud infrastructure – migrated from Oracle Cloud to Microsoft Azure over the course of the project – and evaluated both functionally and against the competitive landscape it was designed to improve on.

Taken together, this work shows that a single, security-conscious microservices platform can plausibly serve both sides of adolescent mental health support – self-care and professional therapy – rather than requiring a

teenager to use two disconnected products.

5.2 Limitations

Three limitations should be kept in mind when interpreting these results. First, this is a software engineering thesis, not a clinical study: no claims are made about the platform’s clinical efficacy, and features that would require clinical judgment were explicitly excluded rather than approximated. Second, the security hardening performed was an internal, structured audit rather than an independent third-party penetration test. Third, the deployment evaluated in Chapter 4 runs on a single virtual machine and has not been load-tested at a scale representative of real-world production traffic.

5.3 Future Work

Building directly on the limitations above, future work on uMatter falls into three groups:

1. **Clinical collaboration.** Any safety-oriented feature that acts on self-reported mental-health data – most notably automated risk detection for at-risk users – requires input from licensed mental-health professionals before it can be responsibly designed, and is intentionally left out of this thesis pending that collaboration. A layered risk-detection framework, informed by clinical guidance, is a natural next step once that collaboration is in place.
2. **Safety-oriented features more broadly.** Beyond risk detection, this includes guardian-visibility controls appropriate for a teenage user base, clearer in-app pathways to crisis resources, and clinical review of the AI companion’s behavior in sensitive conversations.

3. **Operational maturity.** Independent security review beyond the internal audits performed here, load and scalability testing of the current single-VM deployment, and a multi-instance or managed-database deployment topology to remove the single VM as a point of failure.

Pursued together, these directions would move uMatter from a functioning thesis prototype toward a platform that could be responsibly evaluated with real teenage users under appropriate clinical and institutional oversight.

List of Author's Publications

None to date.

References

English

- [1] World Health Organization, *Mental health of adolescents*, WHO fact sheet, 2021. [Online]. Available: <https://www.who.int/news-room/fact-sheets/detail/adolescent-mental-health> (visited on 07/13/2026).
- [2] Daylio. “Daylio – micro diary and mood tracker.” (), [Online]. Available: <https://daylio.net> (visited on 07/13/2026).
- [3] Bearable. “Bearable – symptom and mood tracker.” (), [Online]. Available: <https://bearable.app> (visited on 07/13/2026).
- [4] BetterHelp. “BetterHelp – online therapy & counseling.” (), [Online]. Available: <https://www.betterhelp.com> (visited on 07/13/2026).
- [5] Talkspace. “Talkspace – online therapy & psychiatry.” (), [Online]. Available: <https://www.talkspace.com> (visited on 07/13/2026).
- [6] Wysa. “Wysa – ai-powered mental health support.” (), [Online]. Available: <https://www.wysa.com> (visited on 07/13/2026).
- [7] OWASP Foundation, *OWASP top 10:2021 – the ten most critical web application security risks*, OWASP Foundation, 2021. [Online]. Available: <https://owasp.org/Top10/> (visited on 07/13/2026).
- [8] S. Newman, *Building Microservices: Designing Fine-Grained Systems*, 2nd. O’Reilly Media, 2021.

- [9] R. T. Fielding, “Architectural styles and the design of network-based software architectures,” Ph.D. dissertation, University of California, Irvine, 2000.
- [10] Broadcom Inc. “RabbitMQ – reliable messaging with erlang.” (), [Online]. Available: <https://www.rabbitmq.com> (visited on 07/13/2026).
- [11] VMware Tanzu. “Spring boot reference documentation.” (), [Online]. Available: <https://docs.spring.io/spring-boot/index.html> (visited on 07/13/2026).
- [12] Docker Inc. “Docker – accelerated, containerized application development.” (), [Online]. Available: <https://www.docker.com> (visited on 07/13/2026).
- [13] M. B. Jones, J. Bradley, and N. Sakimura, *JSON web token (JWT)*, IETF RFC 7519, 2015. [Online]. Available: <https://www.rfc-editor.org/rfc/rfc7519> (visited on 07/13/2026).
- [14] Google DeepMind, *Gemini: A family of highly capable multimodal models*, Technical report, 2024. [Online]. Available: <https://arxiv.org/abs/2312.11805> (visited on 07/13/2026).
- [15] Jitsi. “Jitsi Meet – secure, simple, and scalable video conferencing.” (), [Online]. Available: <https://jitsi.org> (visited on 07/13/2026).

Appendix A

API Endpoint Summary

This appendix lists a representative subset of the REST endpoints exposed by each uMatter service. It is intended as a quick-reference companion to the full API reference maintained in the project documentation, not a complete specification.

Table A.1: Representative endpoints by service

Service	Example endpoint	Purpose
Auth	POST /api/auth/login	Authenticate a user and issue a JWT access/refresh token pair
Tracking	POST /api/tracking/mood	Record a mood/sleep/journal entry for the authenticated user
AI	POST /api/ai/chat	Send a message to the AI companion, grounded in the user’s tracking context
Dashboard	GET /api/dashboard/summary	Aggregate tracking data into a user-facing summary view
Therapist	POST /api/therapist/appointments	Book an appointment with a matched therapist
Social	GET /api/social/feed	Retrieve a user’s peer social feed
Notification	GET /api/notifications	Retrieve a user’s in-app notification inbox

Appendix B

Deployment Configuration Notes

This appendix summarizes the cloud deployment topology referenced in Chapter 4: a single Ubuntu virtual machine running the full container stack (application services, PostgreSQL instances, RabbitMQ, Redis, MinIO, and pgAdmin) behind a reverse proxy that terminates HTTPS, with DNS provided by a dynamic DNS service. Secrets and environment files are kept outside version control and are managed directly on the deployment host and in the team's private configuration store.